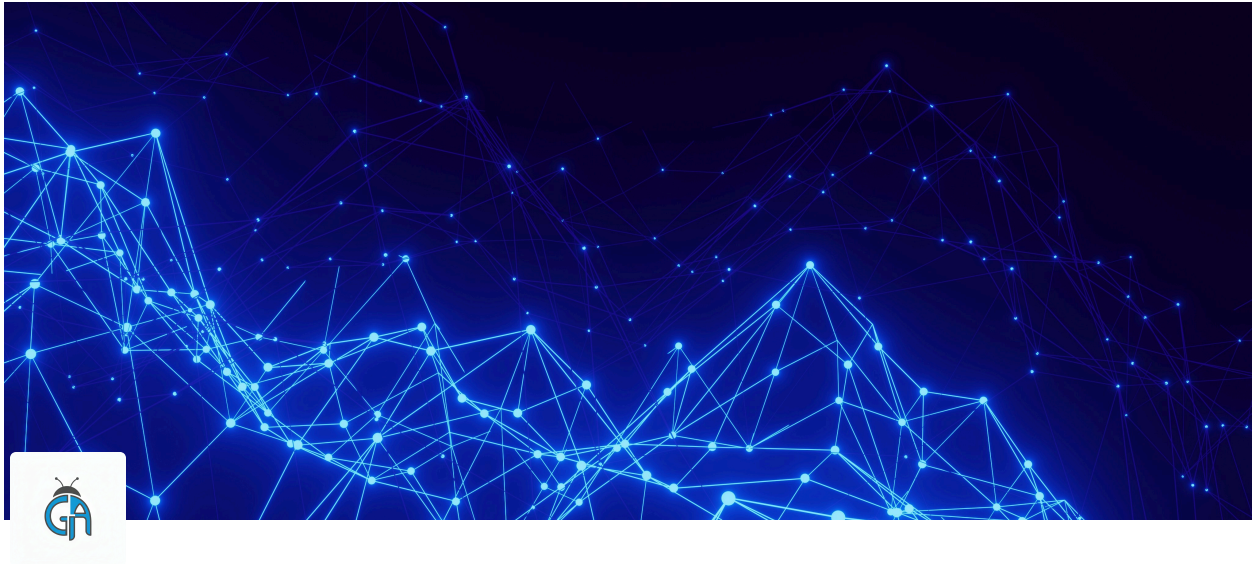




🔥 Errors, Defects, & Failures

The Chain Reaction

👉 **Author:** Georgia Antoniou

🏷️ **Tags:** QA, ISTQB, Definitions

🕒 **Reading Time:** 3 min

🚀 TL;DR

It is a domino effect.

Error: The human mistake (The cause).

Defect: The bug in the code (The result of the mistake).

Failure: The software crashing (The result of the bug).

Root Cause: The reason the human made the mistake.

🧠 The Chain of Events

A "bug" doesn't just appear. It travels through stages:

1. Error (Mistake):

- - *What:* A human action that produces an incorrect result.
 - *Who:* The Developer, Analyst, or Designer.
 - *Example:* A developer forgets a semicolon `;`.

2. Defect (Fault/Bug):

- - *What:* The flaw in the code or document caused by the Error.
 - *Where:* Inside the software (static).
 - *Example:* The logic `if (x > 10)` is written instead of `if (x >= 10)`.

3. Failure:

- - *What:* The software stops doing what it is supposed to do.
 - *Where:* Visible to the user (dynamic).
 - *Example:* The system crashes when the user enters "10".

The Cake Analogy

The Root Cause:

The recipe book has a coffee stain covering the word "Sugar."

The Error (Human):

The Baker (Developer) thinks it says "Salt" and pours salt into the bowl.

The Defect (Code):

The cake mix (System) now contains Salt instead of Sugar.

Note: The cake looks fine sitting on the table. The defect is hidden.

The Failure (User Experience):

The Customer eats the cake and spits it out because it tastes terrible.

The defect has now become a failure.

Root Cause Analysis (RCA)

Finding the bug isn't enough. We need to find the **Root Cause** to prevent it from happening again.

We often use the "**5 Whys**" technique.

The Failure: Application crashed.

1. **Why?** Database timed out. (Defect).
2. **Why?** The query was too slow. (Technical Cause).
3. **Why?** The developer didn't index the table. (Error).
4. **Why?** The developer didn't know they needed to.
5. **Why? Root Cause:** Lack of training on database standards.

Fixing the code solves today's bug. Training the developer solves tomorrow's bugs.

Real World Example

Scenario: A bank calculation.

1. **Error:** The developer is tired and types `+` instead of `-`.
 2. **Defect:** The code line says `Balance = Balance + Withdrawal`.
 3. **Failure:** I withdraw \$50, but my account balance goes **UP** by \$50.
-

Why This Matters

If I report a **Failure** ("The system crashed"), that helps the user.

If I find the **Defect** ("Line 40 is wrong"), that helps the developer.

If I find the **Root Cause** ("Our requirements are confusing"), that helps the whole company.